Taller: Grafos y Recorridos

Objetivo:
Representar un grafo con lista de adyacencia, recorrerlo con BFS o DFS y usar el recorrido para resolver dos problemas clásicos: componentes conexas y detección de ciclos. En cada problema se pide describir el recorrido, su postcondición, su condición de parada y su complejidad antes de implementarlo en Python leyendo la entrada estándar.
Los casos de prueba de cada problema están en la carpeta pruebas/ (amigos.in y amigos.out; ciclo.in y ciclo.out). Su programa debe producir exactamente la salida del archivo .out cuando lee el archivo .in por la entrada estándar.

Problema 1: Grupos de Amigos

Contexto y propósito:

En un pueblo de N habitantes se sabe de algunas parejas que son amigas. Como «los amigos de mis amigos son mis amigos», la amistad se propaga: si A es amigo de B y B de C, entonces A y C están en el mismo grupo aunque nunca se hayan visto. Es el problema Friends (UVa 10608) y aparece, con otros nombres, cada vez que una relación simétrica se propaga: redes sociales, computadores que comparten una red, piezas que se tocan. Lo que se pretende es que reconozca que cada grupo es una componente conexa del grafo de amistades (guía, ejercicio 3), que elija un recorrido que no dependa de la pila de Python (N llega a 30 000) y que lea la entrada de forma eficiente (M llega a 500 000 parejas, y una pareja puede venir repetida).

Enunciado:
Dados N habitantes, numerados de 1 a N, y M parejas de amigos, determine cuántas personas tiene el grupo de amigos más grande. Dos personas están en el mismo grupo si son amigas o si hay una cadena de amistades que las une. Una persona sin amigos forma un grupo de tamaño 1.

Entrada:
- Un entero T, el número de casos de prueba
- Para cada caso: una línea con N y M ($1 \le N \le 30\,000$, $0 \le M \le 500\,000$), seguida de M líneas con dos enteros A y B ($1 \le A, B \le N$, $A \ne B$) que indican que A y B son amigos. Una misma pareja puede aparecer más de una vez.

Salida:

- Para cada caso, una línea con el tamaño del grupo de amigos más grande

Ejemplo:
Entrada:
2
3 2
1 2
2 3
6 3
1 2
1 2
4 5
Salida:
3
2

1. Describa el recorrido: ¿desde qué habitantes arranca, qué estructura usa (cola o pila) y qué marca deja en cada habitante? Explique por qué un habitante que no tiene marca cuando le llega su turno inicia necesariamente un grupo nuevo, y no uno ya contado.

2. Escriba la postcondición del recorrido que arranca en el habitante s: ¿qué garantiza sobre las marcas cuando termina? Indique cómo se obtiene de ahí el tamaño del grupo de s y, al final, el tamaño del grupo más grande.

3. Explique la condición de parada de cada recorrido y por qué el programa completo termina: ¿cuántas veces puede entrar un habitante a la cola? ¿Qué pasaría si se marcara al sacarlo en lugar de al meterlo?

4. Calcule la complejidad en función de N y M: cuente cuántas veces se atiende cada habitante y cuántas veces se revisa cada pareja. Explique por qué el total es O(N + M) y no O(N · M) aunque se hagan muchos recorridos. ¿Qué le pasa a un DFS recursivo en Python si los 30 000 habitantes forman una sola cadena de amistades?

5. Implemente la solución en Python (leyendo la entrada estándar con el formato de Entrada) y pruébela con pruebas/amigos.in: su salida debe coincidir con pruebas/amigos.out.

Problema 2: Senderos Circulares

Contexto y propósito:

Un parque natural tiene n miradores unidos por m senderos; cada sendero se puede caminar en los dos sentidos y entre dos miradores hay a lo sumo un sendero. La administración quiere ofrecer una ruta circular: salir de un mirador, caminar por senderos distintos y volver al mismo mirador sin repetir ninguno. Decidir si existe tal ruta es decidir si el grafo tiene un ciclo. La misma pregunta aparece al verificar que una red de dependencias no sea circular o, en la presentación 7, cada vez que Kruskal decide si una arista se puede agregar al árbol de recubrimiento. Lo que se pretende es que descubra qué información adicional debe llevar el recorrido (desde qué mirador se llegó a cada uno) para reconocer una arista que cierra un ciclo, que note por qué la restricción de un solo sendero por pareja hace falta para que ese reconocimiento sea correcto, y que compare el recorrido con un argumento de conteo: un grafo sin ciclos con k componentes tiene exactamente n − k aristas (guía, ejercicio propuesto 8).

Enunciado:
Dados n miradores, numerados de 0 a n − 1, y m senderos, cada uno entre dos miradores distintos y sin senderos repetidos, determine si existe una ruta circular: una secuencia de tres o más miradores distintos en la que cada par de miradores consecutivos está unido por un sendero y el último está unido con el primero.

Entrada:
- Varios casos de prueba. Cada caso empieza con una línea con n y m ($1 \le n \le 100\,000$, $0 \le m \le 200\,000$), seguida de m líneas con dos enteros u y v ($0 \le u, v < n$, $u \ne v$) que indican que hay un sendero entre u y v.
- La entrada termina con una línea «0 0», que no se procesa.

Salida:
- Para cada caso, una línea con SI si existe una ruta circular o NO si no existe

Ejemplo:
Entrada:
4 4
0 1
1 2
2 3
3 1
5 4

0 1
1 2
2 3
3 4
0 0
Salida:
SI
NO

1. Describa el recorrido: además de la marca de visitado, ¿qué debe recordar de cada mirador y por qué? Indique con precisión qué condición, al revisar un sendero (u, v) desde u, indica que hay un ciclo, y por qué un sendero hacia el mirador desde el que se llegó a u no cuenta.

2. Escriba la postcondición del recorrido que arranca en s cuando termina sin encontrar ciclo: ¿qué cumplen todos los senderos de la componente de s? Explique por qué hay que repetir el recorrido desde cada mirador sin marca y qué se concluye si todos terminan sin ciclo.

3. Explique los casos base y la condición de parada: en qué momento el programa puede detenerse antes de recorrer todo el grafo, y cuándo termina si no hay ciclo. Dé un ejemplo con dos senderos entre los mismos miradores que muestre por qué, sin la restricción de un solo sendero por pareja, no bastaría con ignorar el mirador de llegada.

4. Calcule la complejidad del recorrido en función de n y m. Luego explique el segundo método, que no recorre sino que cuenta: si k es el número de componentes, hay ciclo si y solo si $m > n - k$. ¿Cómo obtendría k? ¿Tiene mejor complejidad que el recorrido?

5. Implemente la solución en Python (leyendo la entrada estándar con el formato de Entrada) y pruébela con pruebas/ciclo.in: su salida debe coincidir con pruebas/ciclo.out.